웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

이 양식은 본인이 구현한 결과를 설명하기 위한 문서입니다. 대괄호 안내를 실제 내용으로 바꾸고 작성 안내 문장은 제출 전에 지우세요. 화면 캡처와 표의 행은 필요에 따라 추가할 수 있습니다. 6~10 쪽 내외를 권장하며 분량 자체는 평가하지 않습니다.

01 서비스 소개와 구현 범위

학번 202212971 이름: 유기태 제출일: 2026/10/28

서비스 이름 오늘 뭐먹지? 메뉴 추천 게시판

기술 조합과 버전 Vue, Spring Boot, jdk21, SQLite

GitHub 저장소 URL https://github.com/ililil-gif/jwt

소스 코드 기준 [브랜치 / 코드 구현 기준 커밋 ID 를 기록할 수 있습니다. 최종 제출 기준 커밋 SHA 는 LMS 에 제출합니다.]

서비스의 대상 사용자와 사용 목적을 3~5 문장으로 설명하세요. 이 기술 조합을 선택한 이유와 제공 템플릿에서 변경한 범위를 적으세요.

항상 오늘은 어떤 메뉴를 먹을지 고민이 되는 학생들을 위한 메뉴를 추천해주는 게시판이다. 누구나 메뉴를 게시판을 통해 천하여 고민을 덜어주는 목적이다. 특히 자취를 하는 학생들은 메뉴를 고민하므로 해당 학생들 중심의 사용자 풀이다.

백엔드는 저번 과제가 Spring boot 이었기 때문에 이번에도 spring boot 로 과제를 진행해 보자 라고 생각하여 선택하게 되었다. 프론트는 이미 개발환경에서 설치되어 있었기에 바로 사용이 가능하여 vue 를 선택하였다.

제공된 템플릿에서 변경한 범위는 기본적으로 템플릿에 주어진 것을 활용하였고 todo 위주로 변경하였다. 백엔드의 토큰발급(issuetoken),토큰검증(currentUserId), 작성자 검사(assertOwner) 범위와 프론트의 로그인 처리(login), 토큰을 헤더로 전송(authorizationHeader), 401 발생 시 로그아웃(request) 범위이고 app.vue 에서 서비스 이름 등과 같은 배너를 바꾸었다.

완료 여부는 완료·부분·미완료 중 하나로 기록하세요.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	BoardController.java()	토큰 없이도 조회 가능
로그인과 JWT 발급	완료	BoardController.java/login() AuthService.java/issueToken()	유효시간은 90 초. Alice 와 bob 계정 사용
인증과 내 정보 조회	완료	AuthService.java/currentUserId() Api.js/authorizationHeaders()	서명, 만료, 검증 등

기능	완료 여부	관련 파일	비고 또는 제한
본인 글 작성·수정·삭제	완료	AuthService.java/assertOwner()	403 검사와 화면버튼 숨김을 적용
만료·로그아웃·오류 표시	완료	Api.js/request() Api.js/clearSession() App.vue/logout()	401 이면 로그아웃과 메시지 출력
DB 데이터 유지	완료	PostStore, data/board.sqlite	서버 재 시작 한 후 글 유지

구조와 변경 내역

요청 흐름, 토큰의 이동, 변경한 파일의 역할을 연결해 설명하세요.

브라우저 → API 서버 → SQLite 의 연결 구조를 간단히 그리거나 글로 설명하세요. 로그인 요청과 게시글 수정 요청의 흐름을 구분하고, 토큰이 어디에 저장되고 어느 요청에서 전송되는지 표시하세요.

웹에서 vue 화면 이 동작하고 화면이 보내는 요청은 spring boot api 서버로 전달된다. api 서버는 글의 저장과 조회를 SQLite 파일 에서 처리한다. 그러므로 요청은 브라우저->api 서버->SQLite 의 순서로 가게 된다.
로그인 요청의 흐름은 사용자가 이메일과 비밀번호를 입력하면 login()이 POST /api/auth/login 을 보내게 되는데 api 서버의 BoardController.login()이 입력 형식을 검사하고 Users.verify()로 계정을 확인한다. 계정이 확인되면 AythService.issueToken()이 jwt 를 만들고 틀리면 401 를 응답한다. 계정은 서버 메모리에 있기에 DB 에는 가지 않는다. 응답을 받은 login()은 토큰과 사용자 정보를 메모리 변수 session 변수에 저장한다.
계시글 수정 요청의 흐름은 먼저 put /api/posts/id 가 전송이 되고 request()가 저장한 토큰을 헤더로 붙인다. 이후 currentUserId()로 토큰을 검증해 요청한 사용자 번호를 얻고 SQLite 에서 해당 글을 찾은 후 assertOwner()을 통해 요청한 사용자와 작성자가 같은지 확인한다. 통과하면 SQLite 의 글을 수정하고 200 으로 응답한다. 즉 토큰검증->SQLite 조회->권한 검사->SQLite 수정 이러한 순서를 거치게 된다.
토큰의 저장 장소와 전송 같은 경우 토큰은 메모리의 session 에 저장된다. 그러기에 새로고침을 하면 로그아웃 되어버린다.토큰의 전송은 로그인 후 글 등록, 수정, 삭제 때 헤더로 전송할게 된다.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
Authservice.java/issueToken()	501 NOT_IMPLEMENTED 를 던짐	서명 및 jwt 발급	로그인 할 때 토큰을 발급받기 위해 ai 초안 및 작성
Authservice.java/currentUserId()	사용자 1 번을 반환	헤더읽기, 서명, 만료, 사용자 검증, 사용자 확인 실패하면 401	서버가 요청자를 확인하기 위해서 ai 초안 및 작성
Authservice.java/assertOwner()	검사를 하지 않음	요청자와 작성자를 비교. 다르면 403	자신의 글만 수정 및 삭제하기 위해 ai 초안 및 작성
Authservice.java/singingKey(), readBearer(), unauthorization()	새로 추가	키의 형식 검사, 헤더에서 토큰 부분을 꺼냄, 401 오류 생성	토큰 발급과 검증에서 동일하게 사용하는 키 변환, 헤더 읽기, 401 오류의 중복을 줄이기 위해
Api.js/login()	예외만 던짐	로그인의 응답의 토큰과 user 을 session 에 저장하고 user 반환	로그인 상태를 사이트에 연결 ai 초안 및 작성

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
Api.js/authorizationHeaders()	빈 객체 반환	토큰이 있으면 헤더 생성	토큰검사를 받는 api 호출에 토큰 전달. ai 초안 및 작성
Api.js/request()	401	401 일 경우 clearSession() 으로 로그아웃 시킴	만료되거나 무효된 토큰인 상태로 계속 있지 못하게 하기 위해 ai 초안 및 작성
Api/canEdit()	True	로그인 사용자와 글쓴이 비교	본인 글에만 수정 및 삭제 버튼 표시하기 ai 초안 및 작성
App.vue	웹의 원래 타이틀 및 베너	배너의 문구 변경	원하는 게시판으로 만들기 위해 ai 초안 및 작성

각 변경의 작성 출처도 구분하세요. 예: 제공 코드 유지, 본인 작성, AI 초안 후 본인 수정, 외부 예제 참고. 보조 파일을 추가했다면 함께 기록하세요.

로컬 실행 기록

실제 설치·빌드·실행 명령과 결과를 적고, 자세한 재현 방법은 저장소 README 에서 연결하세요.

[운영체제와 버전 / 새 폴더 설치 / 환경설정 / 프론트 빌드 / 백엔드 실행 / SQLite 파일 경로]
JDK 21.0.12, Node.js v24.21.0, npm 11.19.0, Edge
실행명령: intellij 에서 BoardApplication 을 실행. 프론트는 테스트 시에 npm run dev 로 실행하여 확인하였다. 최종 화면 테스트는 npm run build 후 npm run preview 에서 수행
환경설정: 64 자리 16 진수 난수를 생성해 JWT_SECRET 으로 설정. 유효시간은 90 초로 설정하고 테스트 할 때 600 초, 만료 테스트는 3 초로 변경 후 테스트 하였다.
SQLite 파일: backend-spring/data/board.sqlite

[4173 번 접속 및 API 연결 확인 / 서버 재시작 후 데이터 유지 / 증거 위치]
http://127.0.0.1:8000/api/health 가 동작하는 것을 확인한 후 4173 번 접속 후 로그인. network 에서 /api 요청이 백엔드인 8000 번으로 전달됨. 글을 작성 후 서버를 재시작 했을 때도 글의 작성자, 내용 등이 그대로 유지됨. 증거위치는 T02_로그인_화면, E1_토큰전송_Authorization 헤더, T15_재시작전_목록, T15_재시작후_목록 의 사진파일에 존재

핵심 코드 설명

네 부분을 각각 설명하세요. 전체 파일을 붙이지 말고 핵심 코드 5~20 줄 정도를 발췌한 뒤 입력 → 처리 → 출력과 실패 조건을 자신의 말로 설명합니다. 성공·실패 시 HTTP 상태와 JSON code, 해당 분기를 만드는 코드도 연결해 적으세요. 줄 번호를 적는 경우 제출한 코드 기준인지 확인하세요.

3 1 로그인과 JWT 발급

파일 / 함수:

계정확인: BoardController.java/login()

토큰생성: AuthService.java/issueToken()

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

BoardController.java login()

```
@PostMapping("/auth/login")
public Object login(@RequestBody Map<String, Object> body) {
  var input = fields(body, Map.of("email", 254, "password", 128));
  var user = users.verify(input.get("email"), input.get("password"));
  if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");//
  return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);
}
```

AuthService.java issueToken()

```
public String issueToken(Users.PublicUser user) {
  Instant now = Instant.now();
  return Jwts.builder()
      .subject(String.valueOf(user.id()))
      .issuer(issuer)
      .audience()
      .single(audience)
      .issuedAt(Date.from(now))
      .expiration(Date.from(now.plusSeconds(ttl)))
      .signWith(signingKey(), Jwts.SIG.HS256)
      .compact();
}
```

동작과 응답 코드 설명

[입력 검증, 계정 확인, 토큰 클레임과 유효 시간, 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

입력검증: 클라이언트가 서버로 {email, password}를 json 의 형태로 서버에 보낸다. 이후 fields()가 올바른 입력인지 확인한다. (body 가 비었는지 문자열인지 길이, 이메일과 비밀번호만 받는지) 잘못되었다면 400 BAD_INPUT, JSON 형식으로 {code, message}로 응답.

계정확인: 코드 안에 작성되어 있던 verify()함수를 통해 이메일로 계정을 찾은 후 입력된 비밀번호를 해시로 바꿔 저장된 해시와 비교한다. 이는 서버가 비밀번호 원문을 가지지 않도록 하기 위해 해시로 비교한다. 일치하면 사용자정보 PublicUser(id, name, email)를 돌려주고 일치하지 않는다면 null 을 돌려준다. 만약 null 이라면 401 {INVALID_CREDENTIALS}로 응답하게 된다.

JWT 토큰: 계정이 확인된 경우에 issueToken()이 호출되어 jwt 토큰이 만들어지게 된다. 서버의 비밀키로 HS256 서명을 한다. 비밀번호는 토큰에 넣지 않는다. 사용자 id 로 사용자가 누구인지 알고 있기에 비번을 토큰에 넣지 않음. 그리고 HS256 의 비밀키는 서버에 두고 같은 키로 발급 및 검증을 한다. Jwt 는 header, payload, signature 이 세 부분으로 나뉜다. 헤더에는 서명알고리즘 명시, 페이로드는 사용자 번호, 토큰의 만료시간, 발급시간, 발급자, 사용하는 곳 그리고 signature 에는 지정된 비밀키로 서명한 값. 무결성을 보장하여 데이터가 변경되지 않았음을 알려준다.

응답: 성공하면 200 ok 와 함께{accessToken, expiresIn, user}를 응답하게 된다. 로그인에 성공하여 토큰을 발급해 준다 라는 의미

로그인 성공 200
이메일, 비밀번호 외의 필드를 보냄 400 { "BAD_INPUT", "허용된 JSON 필드만 보내세요." }
이메일, 비밀번호가 아닌 항목이 옴 400 { "BAD_INPUT", "문자열을 입력하세요." }
이메일이나 비밀번호를 빼먹거나 글자가 아닌 값을 보냄 400 { "BAD_INPUT", " Email(혹은 password)입력을 확인하세요" }
요청형식이 올바르지 않음 400 { "BAD_INPUT", "올바른 JSON 객체를 보내세요." }가입되지 않은 이메일이거나 비밀번호가 틀림 401 { "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요." }

3 2 브라우저 토큰 저장과 전송

파일 / 함수: [작성]
frontend-vue/src/api.js login()
frontend-vue/src/api.js authorizationHeaders(), request()
frontend-vue/src/api.js clearSession()

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

```
let session = { accessToken: "", user: null };
export function clearSession() {
session = { accessToken: "", user: null };
}
function authorizationHeaders() {
const token = session.accessToken;
return token ? { Authorization: `Bearer ${token}` } : {};
}
export async function request(path, options = {}) {
const response = await fetch(`/api${path}`, {
...options,
headers: {
```

```
"Content-Type": "application/json",
...authorizationHeaders(),
...options.headers,
},
});
if (!response.ok) {
if (response.status === 401) clearSession();

}
return data;
}
export async function login(email, password) {
const data = await request("/auth/login", {
method: "POST",
body: JSON.stringify({ email, password }),
});
session = { accessToken: data.accessToken, user: data.user };
return data.user;
}
```

동작과 응답 코드 설명

[로그인 응답 처리, 저장 위치, Authorization 헤더, 로그아웃을 설명] 성공·실패 HTTP 와 JSON code 도적으세요.

먼저 로그인 응답 처리는 login()이 POST /auth/login 에다가 이메일과 비밀번호를 보낸다. 응답으로 응답의 토큰과 토큰의 유효기간 값, 유저를 돌려주고 응답의 토큰과 유저를 session 변수에 저장한다. 토큰의 저장위치는 session 으로 이곳에 토큰과 사용자 정보(id, name, email)를 저장하게 된다. 이곳은 import 문을 통해 처음 불려올 때 메모리 공간에 할당받게 된다. 그러므로 프로그램이 종료될 때 까지만 유지가 된다. Authorization 헤더 같은 경우 request()에서 붙여지게 되는데 request()는 프론트에서 서버로 가는 모든 요청들이 지나가는 공통함수이다. 여기서 authorizationHeaders()에서 session 을 통해 토큰이 있는지 확인하고 있으면 Authorization:Bearer ${token} 헤더를 붙인다. 토큰이 없으면 헤더를 만들지 않는다.
로그아웃은 App.vue 에서 logout()함수에서 clearSession()을 호출하게 되는데 이 함수에서 session 의 토큰과 user 메모리를 비우게 된다. 또한 401 이면 clearSession()을 통해 메모리의 로그인 상태 또한 해제(로그아웃)하게 된다. 그리고 session 은 프로그램이 종료될 때 까지만 유지되므로 새로고침을 하면 session 의 값들이 휘발되어 로그아웃이 된다.

응답코드
로그인 성공 200
잘못된 이메일이거나 비밀번호 401 {“INVALID_CREDENTIALS”, “이메일 또는 비밀번호를 확인하세요. ”}
토큰이 없음 401 {“TOKEN_REQUIRED”, “토큰이 없거나 형식이 다름”}
변조된 토큰 401 {“INVALID_TOKEN”, “올바르지 않은 토큰입니다. ”}
만료된 토큰 401 {“TOKEN_EXPIRED”, “로그인 시간 만료”}
다른 사람 글을 수정 및 삭제시도 403 {“FORBIDDEN”, “본인이 작성한 글만 수정하거나 삭제 할 수 있습니다. ”}

서버 인증과 작성자 권한

아래 설명은 실제 제출한 파일과 함수의 동작을 근거로 작성하세요.

3 3 서버 JWT 검증

파일 / 함수: AuthService.java/currentUserId()

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

```
  var jws =
     Jwts.parser()
        .verifyWith(signingKey())
        .requireIssuer(issuer)
        .requireAudience(audience)
        .build()
        .parseSignedClaims(token);

  if (claims.getExpiration() == null || sub == null || !sub.matches("[1-9][0-9]{0,15}")) {
   throw unauthorized("INVALID_TOKEN", "올바르지 않은 토큰입니다.");
  }

 if (users.find(userId) == null) {
  throw unauthorized("INVALID_TOKEN", "올바르지 않은 토큰입니다.");
 }
 return userId;
```

동작과 응답 코드 설명

[서명·알고리즘·만료·필수 클레임 검사와 사용자 ID 전달, 실패 응답을 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

먼저 readBearer()가 bearer 토큰 형태인지 확인한 후에 토큰 부분을 꺼낸다. 헤더가 없거나 모양이 다를 경우 요청을 거부한다. 다음으로 Jwts.parser()토큰 검사기에서 parrseeSignedClaims(token)을 통해 변조 여부를 확인한다. 서버의 비밀키로 서명을 계산하여 토큰의 서명과 비교하고 이때 발급자와 사용 대상도 확인한다. 서명이 다르거나 발급자, 사용 대상이 맞지 않을 시 거부한다. Hs256 알고리즘을 사용하여 다른 알고리즘 방식은 거부한다. 서명이 정상이어도 exp(만료시간)이 지났으면 만료로 보고 거부한다. 서명이 맞아도 요구하는 값(exp 가 없거나 sub 가 없거나 양의 정수 문자열이 아닌 경우)이 맞지 않으면 거부한다. Sub 를 숫자로 바꾼 후 users.find()를 통해 실제로 존재하는 사용자인지 확인한다. 이러한 검사를 통과하면 사용자 id 를 반환한다.

내 정보 조회 200
새 글 등록 201
글 수정 200
글을 삭제 204

토큰이 없거나 BEARER 토큰 형식이 아님 401 { TOKEN_REQUIRED", "토큰이 없거나 형식이 다름"}
형식불일치, 발급자, 사용처 불일치 401 { “INVALID_TOKEN “, "올바르지 않은 토큰입니다."}
알고리즘이 맞지 않음 401 { "INVALID_TOKEN", "올바르지 않은 토큰입니다."}
exp, sub 없음. 형식의 오류 401 {“INVALID_TOKEN”, “올바르지 않은 토큰입니다.’}
서명은 맞으나 만료됨 401 { "TOKEN_EXPIRED", "로그인 시간 만료”}
해당 번호의 사용자가 없음 401 { "INVALID_TOKEN", "올바르지 않은 토큰입니다."}

3 4 작성자 권한 검사

파일 / 함수: [작성]

코드 발췌

[핵심 코드 5~20 줄을 붙여 넣으세요. 필요한 만큼 영역을 늘립니다.]

```
public void assertOwner(long userId, long authorId) {
 if (userId != authorId) {
   throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정 및 삭제 가능");
 }

}
public Object update(
   @PathVariable String id, @RequestBody Map<String, Object> body, HttpServletRequest req) {
 long user = auth.currentUserId(req), pid = postId(id);
 var p = existing(pid);
auth.assertOwner(user, p.authorId());
var input = fields(body, Map.of("title", 100, "content", 5000));
 return store.update(pid, input.get("title"), input.get("content"));
}
```

동작과 응답 코드 설명

[누구와 누구를 비교하는지, 401 과 403 의 차이, 작성자 위조 방지를 설명] 성공·실패 HTTP 와 JSON code 도 적으세요.

작성자의 권한인 수정 및 삭제의 흐름은 토큰 확인->글의 존재 여부 확인->작성자인지 확인-> 입력 검사->작성자 권한 수행(삭제 및 수정) 순서이다. 먼저 토큰확인(currentUserId)를 통해 jwt 토큰을 검사 한 후 이 요청을 보낸 사용자 번호를 돌려준다. 그 후 existing()를 통해 글의 존재를 확인하는데 글이 존재하지 않을 경우 404 “NOT_FOUND”, “게시글이 없습니다.” 라는 메시지를 응답하게 된다. 이후 assertOwner()를 통해 작성자를 확인하게 된다. 두 값을 비교하게 되는데 요청한 사람의 번호(userid)와 글쓴이의 번호(authorid)를 비교하게 된다. 이 둘을 비교하여 같지 않을 경우 403 “FORBIDDEN”, “본인이 작성한 글만 수정하거나 삭제할 수 있습니다.” 를 응답하게된다. 다음으로 json 이 규칙에 맞는지 검사하게 된다(Fields()). 이후 이러한 절차를 통과하면 수정 및 삭제 가 일어나게 된다. 작성자 위조 방지를 동작하는 부분으로는 create()에서 이루어 진다. 작성자의 id 는 body 가 아니라 currentUserId()를 통해 토큰검사로 돌려 받은 사용자 번호로 작성자의 id 가 정해진다. Body 를 통해 작성자 위조가 일어 날 수 있지만 토큰에서의 사용자번호는 검증을 거친 값이므로 믿을 수 있다. 그리고 filds()에서도 tittle 과 contnet 외의 것 이 들어오면 요청을 거부한다. 401 과 403 의 차이로는 401 은 토큰에 이상이 있을 경우이고 403 은 작성자 확인도 되고 토큰도 확인 했지만 다른 사용자의 글에 손을 댈 경우이다.

내 글을 수정함 200

내 글을 삭제함 204
토큰이 없거나 Bearer 형식이 아님 401 {“TOKEN_REQUIRED”, “토큰이 없거나 형식이 다름”}
토큰의 값이 올바르지 않음 401 {“INVALID_TOKEN”, “올바르지 않은 토큰입니다.”}
토큰은 정상인데 만료 401 {“TOKEN_EXPIRED”, “로그인 시간 만료”}
존재하지 않는 글에 접근할려함 404 {“NOT_FOUND”, “게시글이 없습니다.”}
다른사람의 글에 손대려함 403 {“FORBIDDEN”, “본인이 작성한 글만 수정하거나 삭제할 수 있습니다.”}
tittle, content 이외에 항목을 같이 보냄 400 {“BAD_INPUT”, “허용된 JSON 필드만 보내세요.”}
제목 및 내용이 형식과 맞지 않음 400 {“BAD_INPUT”, “문자열을 입력하세요”} or {“BAD_INPUT”, “title (혹은 content}입력을 확인하세요.”}

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경:
JDK 21, Spring Boot, Node.js v24.21.0, npm 11.19.0, Vue 3.5.43/Edge, API 확인은 Postman, 화면 확인은 Edge/백엔드는 IntelliJ, 프론트는 npm run build 후 npm run preview

로컬 배포 확인: [새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173 번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: [작성]

토큰 TTL:
일반테스트는 600 초, 만료 테스트는 3 초. 기본은 90 초

05_테스트시나리오.md 의 T01~T17 결과를 모두 기록하세요. PASS, FAIL, 미실행을 구분합니다. 예상 결과를 실제 결과로 복사하지 말고 직접 관찰한 HTTP 상태와 JSON code, 화면을 적으세요. 실제 결과 칸에는 예를 들어 403 / FORBIDDEN 처럼 기록합니다. 정상 응답은 200 / 해당 없음처럼 적습니다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	GET 을 이용한 조회 /api/posts /api/posts/10	200	200/해당없음	PASS	T01 목록조회 200 T01 상세조회 200
T02	Post /api/auth/login	200 화면 로그인 표시	200 화면에서는 alice 님 로그인 중	PASS	T02 로그인응답 200, T02 로그인_화면
T03	Post /api/auth/login 틀린 비번과 없는 이메일	401 토큰 미발급 상태	401 INVALID_CREDENTIALS	PASS	T03 틀린비밀번호 401, T03 없는이메일 401
T04	Get /api/me Post /api/posts 토큰 없이 진행할 경우	401	401 TOKEN_REQUIRED	PASS	T04 토큰없음내정보 401, T04 토큰없음글등록 401,T04_헤더
T05	Alice 의 토큰으로 Post /api/posts Get /api/me	201	201/해당없음	PASS	T05 글작성 201, T05 글작성 Location 헤더, T05 내정보조회 200

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T06	본인 글 접근 Put /api/posts/10 Delete /api/11	200 204	200/해당없음 204/해당없음	PASS	T06 본인글수정 200, T06 본인글삭제 204, T06 수정삭제후목록
T07	Bob 의 토큰으로 alice 글 접근 Put /api/post/10	403 원문 그대로 유지	403/FORBIDDEN 200/원문과 동일	PASS	T07_Bob 수정시도 403, T07 수정시도후원문 유지
T08	bob 토큰으로 alice 글 삭제시도 delete /api/posts/10	403	403/FORBIDDEN 200/삭제 후 글 유지 확인	PASS	T08_Bob 삭제시도 403 T08 삭제시도후글유지
T09	Get /api/me Post /api/posts Put /api/posts/10 Delete /api/posts/10	401 변조토큰으로 접근 시도 하지만 변경 x	401/INVALID_TOKEN	PASS	T09 변조토큰내정보 401, T09 변조토큰글등록 401, T09 변조토큰수정 401, T09 변조토큰삭제 401, T09 변조토큰후목록
T10	만료된 토큰으로 접근 Get /api/me put	401 토큰 만료	401/TOKEN_EXPIRED	PASS	T10 만료토큰내정보 401, T10 만료토큰수정 401, T10 만료토큰글등록 401, T10 만료토큰삭제 401, T10 만료토큰후목록, T10 만료화면
T11	Get /api/me	401	401/TOKEN_REQUIRED	PASS	T11 로그아웃후화면, T11 로그아웃후요청 헤더
T12	Post /api/me	400 위조 차단	400/BAD_INPUT 400/BAD_INPUT	PASS	T12_authorId 위조 400, T12_userId 위조 400, T12 위조시도후목록
T13	Post /api/posts	400	400/BAD_INPUT 400/BAD_INPUT 400/BAD_INPUT	PASS	T13 빈제목 400, T13 공백본문 400, T13 제목길이초과 400, T13 입력검증후목록

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T14	Get, put, delete /api/posts/999999 Get /api/posts/qqq	404 존재 하지 않는 정보 접근	404/NOT_FOUND 404/NOT_FOUND 404/NOT_FOUND	PASS	T14 없는글조회 404, T14 없는글수정 404, T14 없는글삭제 404, T14 잘못된 ID_400, T14 토큰없이없는글 수정_401
T15	Post /api/posts Get /api/posts	200	200/해당없음	PASS	T15 재시작전목록, T15 재시작후목록
T16	화면 새로고침	200	200	PASS	T16 새로고침후화면
T17	토큰 없이 Put, delete /api/posts/10 bob 으로 post, put, delete /api/posts	토큰이 없을 경우 401 본인 글 접근은 성공	401/TOKEN_REQUIRED 201/해당없음 200/해당없음 204/해당없음	PASS	T17 토큰없이수정 401, T17 토큰없이삭제 401, T17_Bob 글작성 201, T17_Bob 글수정 200, T17_Bob 글삭제 204

HTTP 응답 확인 기록

예상 코드와 실제 코드를 대조하고, 201·204·401 의 헤더와 본문도 확인하세요.

숫자는 Network 나 API 클라이언트에서 직접 관찰한 값을 기록합니다. 본문에 있는 숫자만으로 판단하지 않습니다. 한 항목이 여러 요청을 포함하면 행을 추가하세요.

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200/해당 없음	T01 상세조회 200, T02 로그인응답 200, T06 본인글수정_200
새 글 등록	**201 및 Location**	201/해당 없음	T05 글작성 201, T05 글작성 Location 헤더
본인 글 삭제	**204 및 빈 본문**	204/해당 없음	T06 본인글삭제 204, T17_Bob 글삭제 204
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400/BAD_INPUT	T13 번제목 400, T13 공백본문 400, T13 제목길이초과 400, T12_authorId 위조 400, T12_userId 위조 400
잘못된 계정	**401 INVALID_CREDENTIALS**	401/INVALID_CREDENTIALS	T03 틀린비밀번호 401, T03 없는이메일 401
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401/TOKEN_REQUIRED	T04 토큰없음내정보 401, T04 토큰없음글등록 401
변조·클레임 오류	**401 INVALID_TOKEN**	401/INVALID_TOKEN	T09 변조토큰내정보_401
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401/TOKEN_EXPIRED	T10 만료토큰내정보_401

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
다른 작성자의 변경	**403 FORBIDDEN**	403/FORBIDDEN	T07_Bob 수정시도 403
없는 글	**404 NOT_FOUND**	404/NOT_FOUND	T14 없는글조회 404

[보호 API 401 의 WWW-Authenticate: Bearer 확인 결과와 증거]

캡쳐: T04 헤더-T04_헤더에서 key 값에 WWW-Authenticate, value 값에 Bearer 이 있는걸 확인 가능하다.

[401 과 403 에서 화면이 각각 어떻게 달라지는지 설명]

401 화면이 되면 로그아웃 되어 화면에서 로그아웃 상태를 표시해 준다. 403 은 토큰은 정상이므로 로그인을 유지하고 실패 메시지를 출력하게 된다.

[실패한 수정·삭제 뒤 공개 조회로 데이터 불변을 확인한 결과]

T07 수정시도후원문유지

수정, 삭제시도 후 공개조회로 데이터의 상황을 살펴본 결과 실패한 요청은 반영되지 않았고 제목과 내용은 데이터가 바뀌지 않고 원래 들어있던 데이터가 유지됨을 확인 가능하다

[토큰 없이 없는 글 수정 401 / 정상 토큰으로 없는 글 수정 404 의 실제 결과]

토큰 없이 PUT /posts/999999 같은 경우 401 TOKEN_REQUIRED 이고 정상 토큰으로 같은 방식으로 해주면 404 NOT_FOUND 이다. 토큰의 여부가 글의 존재 여부보다 먼저 확인한다는 것을 알 수 있다.

T14 토큰없는글수정 401, T14 없는글수정

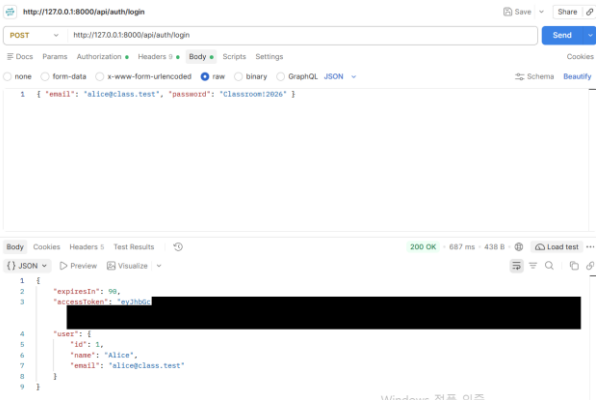
웹서비스설계 04 계속

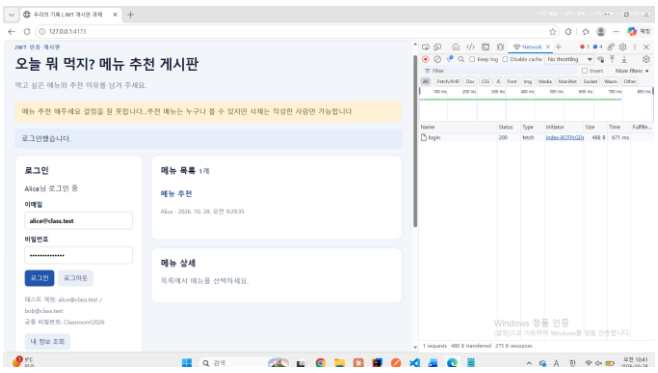
테스트 증거

캡처에는 테스트 ID 와 실제 응답을 표시하세요. 토큰 원문과 서버 비밀 키는 가립니다.

로그인과 토큰 전송

[증거 1: 로그인 성공과 토큰이 첨부된 요청. 토큰 원문은 가리세요.]

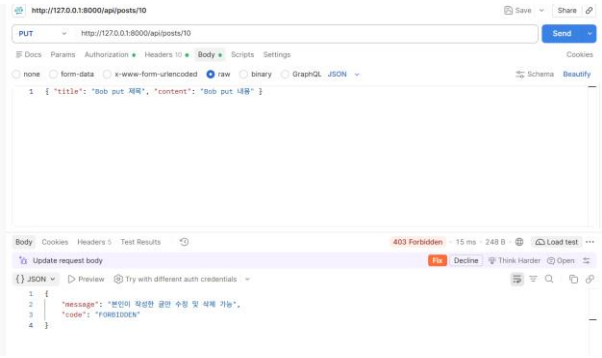




다른 작성자의 수정과 삭제

[증거 2: Bob 이 Alice 의 글 수정·삭제를 시도한 서버 응답]

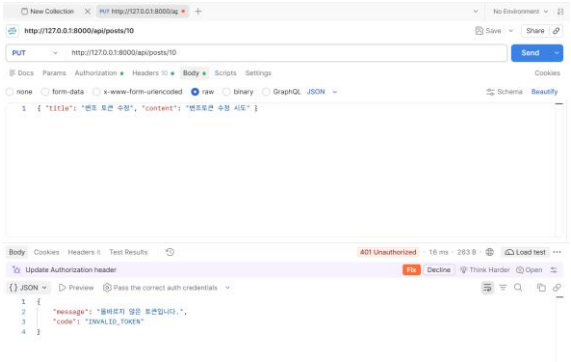
[화면 캡처 또는 요청·응답 삽입]

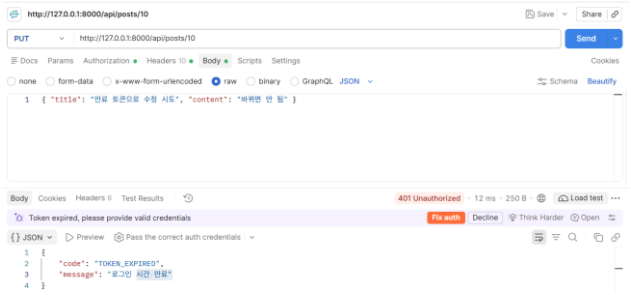


토큰 변조와 만료

[증거 3: 토큰 변조·만료 시 서버 응답과 화면 처리]

[화면 캡처 또는 요청·응답 삽입]





자동 검사와 로컬 배포 증거

[빌드 성공 로그 / 4173 번 API 연결 / 재시작 유지의 화면 또는 파일 경로]

빌드 성공 로그: 백엔드 .\mvnw.cmd package 가 BUILD SUCCESS 로 끝나고 jwt-board-1.0.0.jar 가 생성되었다. 프론트 npm ci 와 npm run build 가 성공했다.

4173 번 API 연결: npm run preview 로 실행한 http://127.0.0.1:4173 에서 로그인하고 글을 작성했다. (4173_로그인_화면.png)

재시작 유지: 백엔드를 종료·재실행해도 작성한 글이 그대로 유지되었다. (4173_로그인_화면.png, T15_재시작전_목록.png, T15_재시작후_목록.png)

AI 활용과 참고 자료

사용 범위, 실제 반영 위치와 직접 확인한 내용을 구분해 기록하세요.

AI 사용 여부: [사용]

구현, 테스트, 보고서 작성 과정에서 AI 를 사용했다. 개념 설명을 듣고 이해하는 용도로 쓰고, JWT 발급, 검증과 권한 검사 코드 초안을 작성하였다. 테스트 절차 설계와 보고서, README 문장 정리에 활용하여 맞는 설명인지 확인 하였으며, 증거 사진 이름 정리, 깃허브 커밋 같은 실행 작업 일부를 대신 수행하였다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
클로드 코드(claude sonnet 5.5)	학습 및 문장 정리	1, 2, 3, 6 절	초안을 통해 학습 후 작성
클로드 코드(claude sonnet 5.5)	코드 생성	AuthServiec.java(BE-01~03, FE-01~04)	로직은 그대로 반영, 주석과 응답 문구는 직접 수정
클로드 코드(claude sonnet 5.5)	테스트 설계	보고서 4 절	수정 후 반영
클로드 코드(claude sonnet 5.5)	문장정리	보고서, 리드미	수정 후 반영
클로드 코드(claude sonnet 5.5)	노가다 대행(증거 캡쳐 이름 변경, 토큰 가리기, git push)	저장소	결과 본인 확인

대표 사례

도구 / 날짜: 클로드 코드/2026-10-24

요청한 내용: BE-01, BE-02, BE-03 을 구현하고 한 줄씩 설명해 줘

AI 가 제안한 내용: issueToken(), currentUserId(), assertOwner()

실제 반영 위치: AuthServiec.java 코드의 동작은 ai 가 작성한 대로 반영함.

내가 확인하고 수정한 내용: 설명을 들으며 각 줄의 역할을 이해했고, 주석과 오류 메시지 문구를 직접 고쳤다. 설명 중 AI 가 틀리게 말한 부분(예: signWith 가 헤더만 만든다는 설명)을 알아내었다.

검증 방법과 결과: [테스트 ID / 요청 / 실제 응답 / 수정 전후 비교]

토큰 없음, 변조, 만료, 다른 작성자의 수정 및 삭제를 Postman 으로 직접 요청해 401 TOKEN_REQUIRED, 401 INVALID_TOKEN, 401 TOKEN_EXPIRED, 403 FORBIDDEN 을 확인했다

도구 / 날짜: Claude Code/2026-10-24

요청한 내용: FE-01~04 를 구현하고 흐름을 설명해 줘

AI 가 제안한 내용: login(), authorizationHeaders(), request()의 401 처리, canEdit()
실제 반영위치: api.js 의 login(), authorizationHeaders(), request(), canEdit()에 반영했다.
내가 확인하고 수정한 내용: 코드의 설명을 듣고 각 줄에 설명 주석을 달았다.
검증 방법과 결과: 브라우저 Network 에서 me 요청에 Authorization: Bearer 헤더가 실리는 것, 로그아웃 후 헤더가 빠지고 401 이 오는 것, 만료 후 화면이 로그아웃 상태로 바뀌는 것을 확인했다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
6 주차 강의 자료 jwt 로그인과 인증	BE-02 이해, 보고서 3.1~3.3	Jwt 구조와 로그인 검증 개념 공부
JWT(JSON 웹 토큰) https://docs.tosspayments.com/resources/glossary/jwt	보고서 3.1 절 (JWT 구성 설명)	JWT 가 헤더·페이로드·서명으로 이루어진다는 개념을 이해하는 데 참고.

이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

요청자가 본문에 authorID 적어서 서버가 이것을 믿어버리면 다른 사람의 이름으로 글을 쓰는 등의 문제가 생긴다.(위조 발생) 이 코드 같은 경우 작성자를 본문이 아닌 auth.currentUserId(req)로 사용자 번호를 정하게 된다. 즉 검증된 토큰을 통해 사용자 번호를 정한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

수정 버튼을 숨기는 것은 postman 같은 도구로 서버에 직접 요청하면 우회됨. 그래서 서버의 AuthService.assertOwner()가 요청자와 작성자의 번호를 비교하여 다르면 403 을 던지게 된다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

Decode 는 토큰의 페이로드를 읽지만 서명은 읽지 않아서 내용을 바꾼 토큰을 만들 수 있다. Verify 는 서버의 비밀키로 서명을 계산하여 토큰의 변조유무를 확인한다. 이 코드는 Jwts.parser().verifyWith(key)…parseSignedClaims(token)으로 서명을 검사하고 iss 와 aud, exp 를 확인한다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 clearSession()을 통해 토큰과 사용자 정보를 지우게 된다. 토큰 만료는 토큰의 사용시간이 지난 토큰을 401 로 거부하고 프론트가 로그인 상태를 해제하게 한다. 새로 고침은 메모리 변수가 초기화 되어 로그아웃 되지만 글은 메모리가 아닌 데이터베이스에 있으므로 그대로 유지된다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

Postman 으로 테스트하려고 토큰 유효 시간을 TOKEN_TTL 로 늘렸는데 로그인 응답의 expiresIn 이 계속 90 으로 나왔다. 코드는 TOKEN_TTL 환경변수를 읽고 없으면 기본값 90 을 쓰는데, 서버에 그 환경변수가 전달되지 않은 것이라고 판단했다. IntelliJ 실행 설정의 환경 변수 칸에 TOKEN_TTL=600 이 제대로 들어갔는지와 서버를 재시작했는지를 확인해 고쳤고, 다시 로그인하자 expiresIn 이 600 으로 나왔다.

현재 남은 문제와 다음에 개선할 점: 새로고침을 해도 일정 시간동안 로그인이 유지되도록 하는 방식으로 개선. 계정이 테스트 계정 두개 밖에 없기에 회원가입이 가능하도록 개선

제출 확인: [ ] 실제 코드와 설명 일치 [ ] 테스트 직접 수행 [ ] AI 활용 및 출처 기록 [ ] 비밀 키와 토큰 제거

GitHub 확인: [ ] 루트 README 작성 [ ] 보고서 PDF 와 증거 업로드 [ ] README 문서 링크 확인 [ ] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출